Laboratory Exercise 02

PicoRV32 on Tang Nano 9K: USB UART, LED control, and synchronization with an educational HTML model

Lab instructions and completed experimental record
Document revision: 1.0 · **Experimental system:** LAB-02 v0.1.0 · **Test date:** 20 October 2026
Project: Sipeed FPGA · **Board:** Sipeed Tang Nano 9K · **FPGA:** GW1NR-9C
Educational experiment: A. A. Malachevsky, independent researcher · ORCID 0009-0008-6009-3196
Upstream CPU RTL: PicoRV32 by Claire Xenia Wolf, ISC license; a pinned upstream revision is used.
Repository: https://github.com/AIDevelopersMonster/Sipeed-FPGA

> **Verified result.** A physical Tang Nano 9K provides bidirectional UART through its existing USB connection (COM10, 115200 8N1). Commands are interpreted by firmware running on PicoRV32, change the MMIO LED register, and are acknowledged over UART TX. A linked HTML teaching model ties **step 14 of 19** to switching on the selected physical LED; stepping back switches it off, and stepping forward switches it on again. Manual and automatic modes were exercised. Status: **MODEL_PASS · RTL_PASS · PNR_PASS · STA_PASS · BOARD_PASS · HTML_HARDWARE_PASS**.

Scope of this document. Sections 1–7 are student instructions; section 8 is the completed record based on command outputs and reported physical observations from the test bench; section 9 is a blank student form. Values not recorded in the evidence, including the final `.fs` SHA256 and physical board serial number, have deliberately not been invented.

1. Objective, learning outcomes, and routes

Objective: extend the previously tested minimal RISC-V SoC with UART receive, control physical FPGA outputs through PicoRV32, and associate a pedagogical execution model of an `SW` instruction with an acknowledged physical effect.

On completion, the student should be able to:

1. Distinguish the PicoRV32 CPU, RV32I firmware in boot ROM, memory-mapped I/O registers, and actual FPGA pins.
2. Explain UART RX commands and UART TX acknowledgments, including why L=n is not an electrical or optical measurement.
3. Reproduce firmware modeling, RTL tests, Gowin PnR/STA and **configuration-SRAM-only** programming.
4. Test LED selection and the **13 → 14 → 13 → 14** reverse/forward sequence in the browser model.
5. Distinguish *event-and-acknowledgment synchronization* from cycle-accurate tracing of the real CPU.

Route A — classroom demonstration: the instructor supplies a previously validated `.fs` image and its SHA256; the student loads it into SRAM, exercises UART and uses the HTML demonstration. Do **not** assume a ready-to-program `.fs` is checked into Git: build artifacts may be excluded from version control.

Route B — engineering workflow (the route recorded here): the student runs the firmware and RTL tests, builds the FPGA image using Gowin, reviews STA, programs SRAM, and performs both serial-port and HTML tests. These stages apply to the same LAB-02 design revision.

2. Hardware and prerequisites

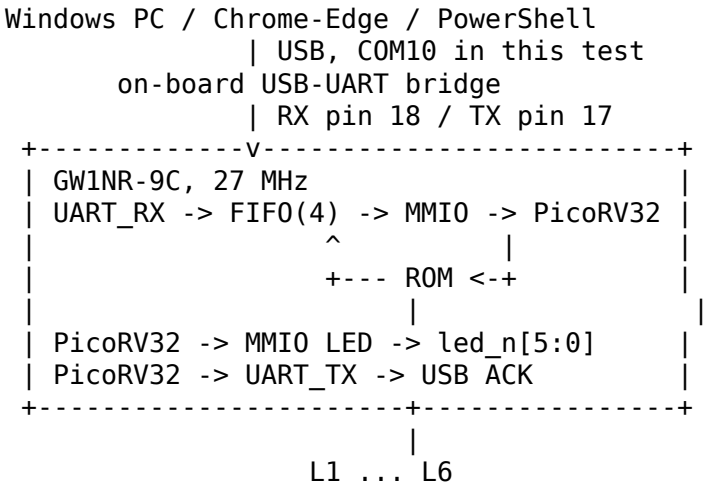
Component	Tested setup / requirement
FPGA board	Sipeed Tang Nano 9K, GW1NR-9C FPGA family
Connection	One standard data-capable USB cable; **no separate USB-TTL adapter**
SoC clock	On-board 27 MHz oscillator, 37.037 ns timing constraint

Component	Tested setup / requirement
Serial device	Board USB-UART bridge; **COM10** in this test, 115200 baud, 8N1
Host	Windows 10, PowerShell 7, Git and Python 3
RTL verification	Icarus Verilog (`iverilog`, `vvp`)
FPGA toolchain	Gowin V1.9.9 Beta-4 Education, `gw_sh.exe`, Programmer CLI
Browser	Chrome or Edge supporting Web Serial, launched from `localhost`

Tested pin constraints: UART TX — FPGA pin 17, UART RX — pin 18; LEDs L1–L6 — pins **10, 11, 13, 14, 15, 16**. The on-board LEDs are *active low*: a logical 1 in `leds[i]` drives `led_n[i]` low, illuminating that LED. No extra signal wiring is required.

Safety and reproducibility: select the correct FPGA device, leave the verified USB data connection intact, do not change I/O voltage levels, release the COM port before connecting another client, and configure **SRAM only**, not `embFlash`. A successful build or test of this revision does not automatically validate later modified RTL or firmware.

3. Architecture and observable behavior



The CPU executes **118 RV32I machine instructions** from a 1 KiB boot ROM (unused words decode as NOP). The SoC has no RAM, stack, C runtime, interrupts, or operating system. UART RX uses two-stage input synchronization, mid-bit sampling, and a four-byte FIFO with overrun and bad-stop-bit flags. This receiver has **no majority-vote oversampling**; send one command at a time and await acknowledgment.

3.1. MMIO address map

Address	Register	Access / meaning
`0x20000000`	LED	R/W, low six bits
`0x20000004`	CYCLES	R/O, free-running counter
`0x20000008`	DEVICE_ID	R/O, `0x5A5A0001` (firmware signature, not JTAG ID)
`0x2000000C`	TEST_STATUS	R/W, retained from baseline SoC
`0x20000010`	UART_TX	low byte written to transmitter
`0x20000014`	UART_STATUS	bit 0 — transmitter ready
`0x20000018`	UART_RX_DATA	reading pops one FIFO byte
`0x2000001C`	UART_RX_STATUS	bit 0 — data available; bit 1 — overrun; bit 2 — framing error; write clears error flags

3.2. Protocol and LED mapping

PC sends	LED mask	Active physical pin	PicoRV32 reply
0	0x00	all pins HIGH, LEDs off	L=0
1	0x01	pin 10 → LOW, L1	L=1
2	0x02	pin 11 → LOW, L2	L=2
3	0x04	pin 13 → LOW, L3	L=3
4	0x08	pin 14 → LOW, L4	L=4
5	0x10	pin 15 → LOW, L5	L=5
6	0x20	pin 16 → LOW, L6	L=6
?	unchanged	read current LED register	L=n

On reset, firmware clears the LEDs and prints READY\n. Each 0–6 command invokes a **real SW write to the LED register**, followed by an LW readback. A match produces L=n\n; a mismatch produces F\n. An unknown character produces E\n (CR/LF ignored). **n is the LED selector, not the bitmask.** The response confirms the *MMIO register readback*, not physical light output; visual inspection is an independent requirement.

4. Route B: reproducible engineering workflow

Run commands from the stated directories. Do not modify the known-good `main` branch; the LAB-02 working branch is `feature/picorv32-uart-console-lab02`. Seven other unrelated untracked student-kit files may exist in the worktree: do **not** delete them or add them indiscriminately with `git add -A`.

Step B1. Sources and build inputs

```
Set-Location C:\GitHub\Sipeed-FPGA
git branch --show-current
git status -sb
Test-Path .\01-PicoRV32\rtl\uart_rx_8n1.v
Select-String -Path .\01-PicoRV32\gowin_build.tcl -Pattern 'uart_rx_8n1.v'
```

Required line in `01-PicoRV32/gowin_build.tcl`:

```
add_file rtl/uart_rx_8n1.v
```

During the actual experiment, Gowin initially raised `EX3937: Instantiating unknown module 'uart_rx_8n1'`. The file was present in `.gprj` but **missing from the CLI's `gowin_build.tcl`**. Adding the line resolved the build. The corrected Tcl file must be included in the eventual Git commit: updating `.gprj` alone does not fix the tested CLI workflow.

Step B2. Firmware model and RTL tests

```
Set-Location C:\GitHub\Sipeed-FPGA
& .\01-PicoRV32\tools\test_lab02.ps1 -ModelOnly
& .\01-PicoRV32\tools\test_lab02.ps1
```

Expected markers: `ASSEMBLE PASS`, `LAB02_FIRMWARE_MODEL_PASS`, `LAB02_RX_RTL_PASS`, `UART TB PASS`, `LAB02_SOC_RTL_PASS`. The firmware model checks RV32I instruction behavior and protocol. The RTL tests check UART bit-level behavior and the real PicoRV32 RTL integrated with the SoC. None of these alone is a physical hardware test.

Step B3. Gowin PnR and static timing analysis

```
& .\01-PicoRV32\tools\build_gowin.ps1
```

The output should include `01-PicoRV32/impl/pnr/picorv32_nano9k.fs`. Open `picorv32_nano9k.tr.html` and check the actual Fmax, setup/hold TNS, and violated endpoints. At 27 MHz, a zero-exit Gowin process alone is insufficient: timing constraints must also be satisfied. Record the device/resource summary and timing corners.

Step B4. Configure SRAM only

Close existing COM terminals. Use the **new** `.fs`, not an arbitrary older bitstream.

```
Set-Location C:\GitHub\Sipeed-FPGA\01-PicoRV32
$prog = 'C:\Gowin\Gowin_V1.9.9Beta-4_Education\Programmer\bin\programmer_cli.exe'
$fs = (Resolve-Path '.\impl\pnr\picorv32_nano9k.fs').Path
Get-Item $fs | Select-Object Name, Length, LastWriteTime
& $prog --device GW1NR-9C --operation_index 2 --fsFile $fs --cable-index 1
"PROGRAMMER_EXIT_CODE=$LASTEXITCODE"
```

In the tested Programmer version, `--operation_index 2` is the validated **volatile SRAM configuration** operation. Do not choose embFlash operations for this exercise. Record the exit code and the binary SHA256 for your own complete report.

Step B5. Board USB-UART and visible LEDs

```
Set-Location C:\GitHub\Sipeed-FPGA
& .\01-PicoRV32\tools\probe_lab02_serial.ps1 -PortName COM10
```

The script rapidly sends `0`, `1`, `4`, `?`, `6`, `0` and checks the replies. It reports `LAB02_SERIAL_ACK_PASS` on success. **The LEDs may appear to flash too quickly to observe** during this automated sequence. For an independent visual check, send one command and wait for Enter before advancing:

```
$sp = [System.IO.Ports.SerialPort]::new('COM10',115200,
  [System.IO.Ports.Parity]::None,8,[System.IO.Ports.StopBits]::One)
$sp.NewLine = "`n"
$sp.ReadTimeout = 2000
try {
  $sp.Open()
  Start-Sleep -Milliseconds 500
  $sp.DiscardInBuffer()
  foreach ($t in @(@('1','L1'),@('4','L4'),@('6','L6'),@('0','ALL OFF'))) {
    $sp.Write($t[0])
    $reply = $sp.ReadLine().Trim()
    Write-Host "$($t[0]) -> $reply; observe $($t[1])"
    Read-Host 'Inspect board; press Enter to continue' | Out-Null
  }
} finally { if($sp.IsOpen){$sp.Close()}; $sp.Dispose() }
```

An L=n acknowledgment must be complemented by direct LED observation. If the test fails, first verify COM number, LAB-02 SRAM image, and exclusive serial-port ownership before editing RTL or changing drivers.

5. Linked HTML practical exercise

Setup: one local HTTP server, one supported browser, the existing USB cable. No external UART adapter, Wi-Fi, Internet service, or flash programming is needed.

```
Set-Location C:\GitHub\Sipeed-FPGA
& .\01-PicoRV32\tools\launch_lab02_html.ps1 -Language EN
```

Open Chrome or Edge at:

`http://127.0.0.1:8765/PicoRV32_LED1_Linked_EN.html`

For the Russian version use `-Language RU` and `PicoRV32_LED1_Linked_RU.html`. `localhost` provides a suitable secure context for Web Serial. The user grants COM-port access through the browser; close any other terminal first.

Task H1. LED selection and steps 1–13

1. Click **Connect COM** and select the board's USB serial port.
2. Select an LED, for example **L4** (bitmask `0x08`, physical pin 14).
3. Advance through **steps 1–13**, observing ROM, x5, x6, and `mem_addr`, `mem_wdata`, `mem_wstrb`, `mem_valid`, `mem_ready`.
4. Until the modeled MMIO acknowledgment at step 14, the selected physical LED should remain off.

Task H2. Step 14 and the reversible teaching action

1. **13 → 14.** HTML sends `'4'`; real PicoRV32 writes and reads LED MMIO and returns L=4; physical L4 lights, and the model shows confirmed state.
2. **14 → 13.** Clicking Back sends `'0'` and waits for L=0; physical L4 turns off.
3. **13 → 14.** Moving forward once more repeats the command; L4 turns on again.
4. Continue to the final steps: the teaching model shows a repeating `JAL x0,0`, while the LED remains on.

Task H3. Automatic mode and multiple LEDs

Run automatic progression from reset; verify that the hardware change is tied to step 14 rather than an arbitrary animation delay. Repeat with several LED selections. When done, disconnect COM and stop the local HTTP server with `Ctrl+C`.

Critical validity boundary. HTML displays a **four-instruction example** (`LUI`, `ADDI`, `SW`, `JAL`) expanded into **19 pedagogical steps**. The physical FPGA instead executes a separate **118-instruction UART command-handling firmware**. The browser does not single-step the real CPU. Moving backward does **not reverse CPU history**; it sends a *new command* to turn the LED off. The synchronized quantity is the *physical-effect event and its acknowledgment*, **not cycle-by-cycle CPU execution**.

6. Completion criteria and review questions

Pass criteria: provide firmware/RTL logs (Route B), the actual `.fs` and its checksum when accessible to the student, STA with zero setup/hold violations, successful UART ACKs, visual on/off transitions for at least three LEDs, and the confirmed **13 → 14 → 13 → 14** HTML sequence. Verification labels remain specific to what was actually tested.

Questions: Why does L=4 select LED L4 while the MMIO write data is 8? What logic level is applied to pin 14? Why is an ACK not optical proof? Why are requests serialized? Why does positive setup slack not prove immunity of an asynchronous UART input under all conditions? What exactly does the HTML Back action do to the actual CPU? How do `.fs` and `boot.hex` differ?

7. Out-of-scope checks

No exhaustive RV32I ISA verification, long-running UART fault/stress evaluation, dense burst/back-to-back command characterization, alternate-board or other Gowin-version qualification, LED current or light-output measurement, `embFlash` programming, cycle-accurate CPU tracing, or actual CPU halt at each of the 19 teaching steps was performed. The STA result applies to constrained `clk27` synchronous paths and is not comprehensive qualification of all asynchronous interfaces.

8. Completed experimental record

Tested: 20 October 2026; Windows/PowerShell 7; Tang Nano 9K (GW1NR-9C), 27 MHz; on-board USB-UART **COM10**, 115200 8N1; Gowin **V1.9.9 Beta-4 Education**. The board serial number and final bitstream SHA256 were not included in the reported console output.

Stage	Recorded observations	Status
Assembler	**118** instructions; **1024-byte** ROM; **22** labels	PASS
Firmware interpreter	**422** executed instructions; READY, L=0, L=1, L=4, L=6, L=6, E, L=0; LED writes `[0,0,1,8,32,0]`	MODEL_PASS
UART RX RTL	Normal bytes, `0x00`, false start, bad stop bit	RTL_PASS
UART TX RTL	Transmitted 0xA5 as UART 8N1	RTL_PASS
Full SoC RTL	READY, commands `1,4,?,0`, invalid command, simulated `led_n` outputs	RTL_PASS

Stage	Recorded observations	Status
Gowin	Fixed missing `uart_rx_8n1.v` entry in `gowin_build.tcl`, then PnR completed successfully	PNR_PASS
Static timing	`clk27` constraint **27.000 MHz**; Actual Fmax **52.522 MHz**; Setup TNS **0**, Hold TNS **0**; zero violated endpoints	STA_PASS
Board UART/MMIO	`0→L=0`, `1→L=1`, `4→L=4`, `?→L=4`, `6→L=6`, `0→L=0`; `LAB02_SERIAL_ACK_PASS`	BOARD_PASS (UART/MMIO)
Physical LEDs	Visually confirmed **L1, L4, L6**, and all off using Enter-paced requests	BOARD_PASS (LED)
HTML/FPGA	Manual 19-step progression, automatic mode, Back = off, Forward = on, repeated for **three** selected LEDs (their IDs were not individually logged)	HTML_HARDWARE_PASS

PnR resource usage: 1868 LUT/ALU/ROM16 = 1512 LUT + 356 ALU; 767/6693 registers (~12%); 1127/4320 CLS (~27%); BSRAM **12%**.
Timing: 2154 paths, 1883 endpoints; worst setup slack **+17.997 ns**, worst hold slack **+0.563 ns**; setup corner Slow 1.14 V / 85 °C, hold corner Fast 1.26 V / 0 °C.
Generated image: `picorv32_nano9k.fs`, **3,558,473 bytes**, file timestamp 20 October 2026, 09:14:59.
Evidence boundary: command outcomes come from the test logs provided in the discussion; visible LED and HTML behavior was reported by the experimenter. A separate synchronized electrical or video trace was not provided.

Conclusion: the specified functional objective has been met. The teaching interface generates acknowledged operations on a real CPU-controlled LED peripheral. Its 19-step execution sequence remains conceptual. LAB-02 v0.1.0 can be classified as completed for the tested scope; publishing the development branch to GitHub is a separate action and is not claimed by this record.

9. Blank student report form

Field	Student entry
Name, course/group	_____
Date, OS / PowerShell	_____
Board type, FPGA, board serial	_____
Git branch / commit ID	_____
Gowin, Icarus, Python versions	_____
COM port / UART settings	_____
`.fs` filename, size and SHA256	_____
MODEL / RTL / PNR / STA readings	_____
UART commands → acknowledgments	_____
Physically observed LEDs	_____
HTML steps 13 → 14 → 13 → 14	_____
Auto mode / additional LEDs	_____
Errors, limitations and conclusion	_____

References and source files

- PicoRV32 SoC source folder; RV32I firmware; UART RX RTL.
- LAB-02 RTL tests; physical serial probe; HTML launcher.
- HTML (English) · HTML (Russian).

- Project repository · Original PicoRV32 CPU.